

Trabalho 2 — Deep Learning aplicado ao MNIST

Inteligência Artificial — Redes Neurais Convolucionais

Felipe Alcantara Martins e Iasmin Laje

29 de maio de 2026

Trabalho 2 — Deep Learning aplicado ao MNIST

Equipe: Felipe Alcantara Martins e Iasmin Laje

Repositório com as soluções da equipe (item f):

<https://github.com/Felipe-Alcantara/Trabalho-2---Deep-Learning-aplicado-ao-MNist>

1. Objetivo

Resolver o **MNIST** (70.000 imagens de dígitos manuscritos 28×28 em tons de cinza) usando três arquiteturas de redes neurais profundas. Partindo da implementação de referência da **LeNet** (marceloarantes19/deepLearning), pesquisamos e implementamos **mais duas** arquiteturas, treinamos todas por **10 épocas** e comparamos suas **matrizes de confusão**.

Conseguimos implementar, treinar e comparar as três soluções com sucesso. A execução oficial foi feita no **Google Colab com GPU** (10 épocas, batch_size=256, otimizador Adam, perda categorical_crossentropy).

2. As três arquiteturas

2.1 LeNet (base de referência)

CNN clássica de LeCun (1998): dois blocos **Conv 5×5 (ReLU) + MaxPooling 2×2** (20 e 50 filtros), seguidos de **Dense 500 (ReLU)** e **Dense 10 (Softmax)**. Simples, rápida e já muito eficaz no MNIST. **1.256.080 parâmetros**.

2.2 CNN Moderna (estilo VGG, com BatchNorm e Dropout)

CNN mais profunda: dois blocos com **duas Conv 3×3 (ReLU) + BatchNormalization** cada, **MaxPooling 2×2** e **Dropout (0,25)**, seguidos de **Dense 256 (ReLU) + BatchNorm + Dropout 0,5** e **Dense 10 (Softmax)**. As convoluções 3×3 empilhadas ampliam o campo receptivo com menos parâmetros; a BatchNormalization estabiliza o treino e o Dropout combate o overfitting. **872.426 parâmetros**.

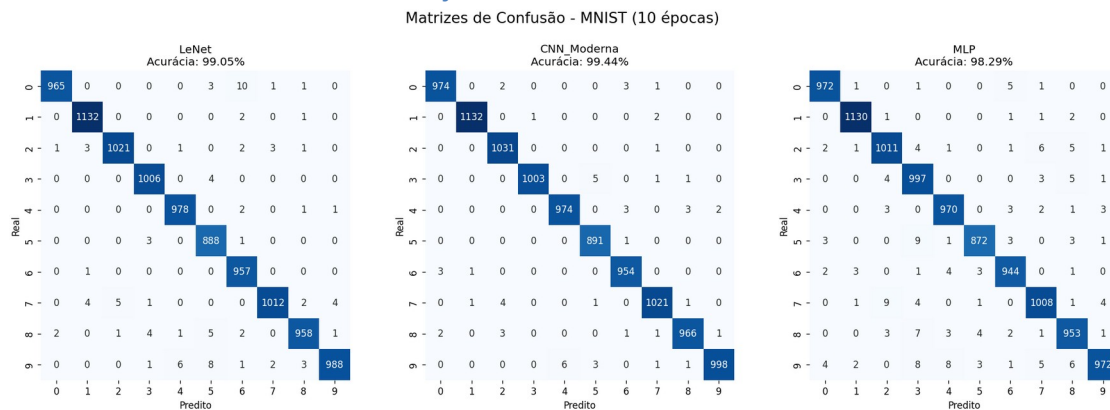
2.3 MLP (Perceptron Multicamadas — linha de base)

Rede totalmente conectada: entrada de **784** (imagem achatada), **Dense 512 (ReLU) + Dropout 0,2**, **Dense 256 (ReLU) + Dropout 0,2** e **Dense 10 (Softmax)**. Ignora a estrutura espacial da imagem, servindo de linha de base. **535.818 parâmetros**.

3. Resultados (item d) — tabela e matrizes de confusão

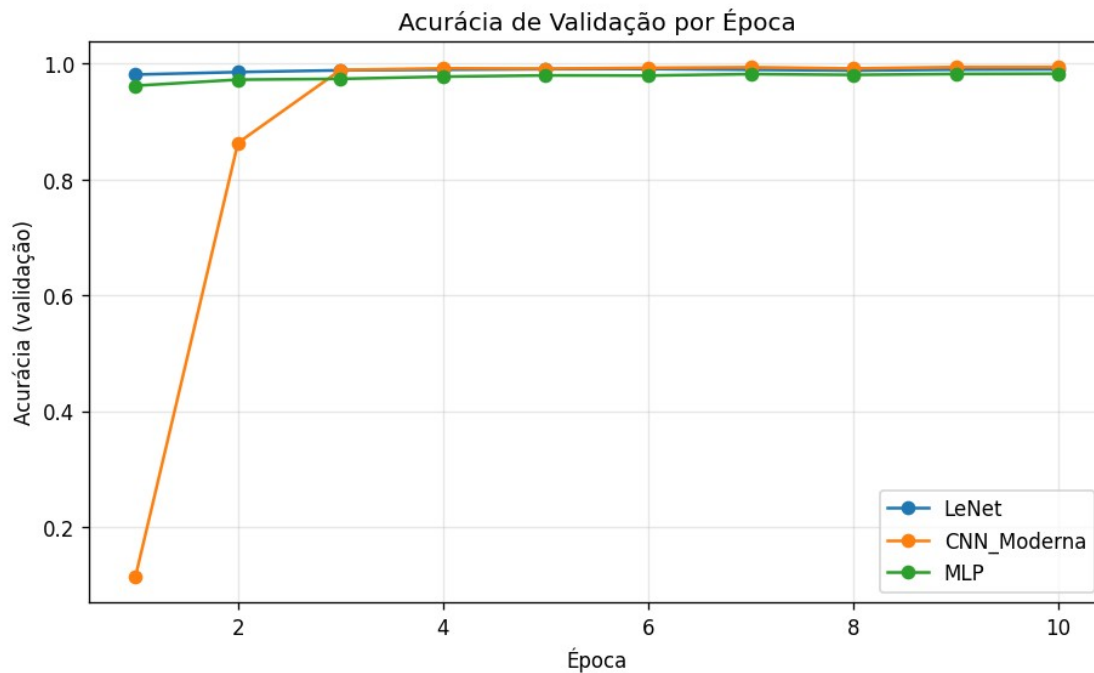
Arquitetura	Parâmetros	Acurácia no teste	Erros (em 10.000)
CNN Moderna	872.426	99,44 %	56
LeNet	1.256.080	99,05 %	95
MLP	535.818	98,29 %	171

Matrizes de confusão das três soluções



Matrizes de confusão das três redes (LeNet, CNN Moderna e MLP).

Acurácia de validação por época



Acurácia de validação ao longo das 10 épocas.

4. Conclusão (item e)

Com base nos resultados observados:

1. **As redes convolucionais superam o MLP com folga.** Mesmo o MLP atingindo 98,29 %, ele comete quase **três vezes mais erros** (171) que a CNN Moderna (56). Isso confirma que **explorar a estrutura espacial da imagem** (convoluções) é decisivo em problemas de visão — o MLP, que trata os pixels como um vetor sem relação espacial, espalha mais erros entre dígitos parecidos (por exemplo $7 \rightarrow 2$, $5 \rightarrow 3$, $9 \rightarrow 3/4/7$).
2. **A CNN Moderna foi a melhor solução** (99,44 %), e ainda com **menos parâmetros que a LeNet**. O ganho vem da maior profundidade (convoluções 3×3 empilhadas) combinada com **BatchNormalization** (treino estável) e **Dropout** (menos overfitting). Sua matriz de confusão é a mais “limpa”, com a diagonal fortíssima e sem nenhuma confusão sistemática grande.
3. **A LeNet teve ótimo desempenho** (99,05 %), confirmando por que continua sendo uma referência didática: arquitetura simples que já resolve o MNIST muito bem.
4. **Curva de aprendizado:** a CNN Moderna começa baixa na 1ª época (efeito da BatchNormalization antes de estabilizar suas estatísticas) e **dispara a partir da 2ª–3ª época**, ultrapassando as demais; a LeNet já nasce alta e estável; o MLP fica consistentemente abaixo das duas CNNs.

Em resumo, para o MNIST em 10 épocas: CNN Moderna > LeNet > MLP. A arquitetura mais moderna entregou a melhor acurácia com o menor número de erros e de parâmetros, ou seja, o melhor custo-benefício.

5. Link do GitHub (item f)

Todas as soluções da equipe (notebooks das três arquiteturas, scripts, resultados e documentação) estão no repositório:

<https://github.com/Felipe-Alcantara/Trabalho-2---Deep-Learning-aplicado-ao-MNist>

*Trabalho realizado por **Felipe Alcantara Martins** e **Iasmin Laje**.*